

```

1 //#####
2 // FILE:   HWstarter_main.c
3 //
4 // TITLE:  HW Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "driverlib.h"
16 #include "device.h"
17 #include "F28379dSerial.h"
18 #include "LEDPatterns.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22
23 #define PI          3.1415926535897932384626433832795
24 #define TWOPI       6.283185307179586476925286766559
25 #define HALFPI      1.5707963267948966192313216916398
26 // The Launchpad's CPU Frequency set to 200 you should not change this value
27 #define LAUNCHPAD_CPU_FREQUENCY 200
28
29
30 // Interrupt Service Routines predefinition
31 __interrupt void cpu_timer0_isr(void);
32 __interrupt void cpu_timer1_isr(void);
33 __interrupt void cpu_timer2_isr(void);
34 __interrupt void SWI_isr(void);
35
36 // Count variables
37 uint32_t numTimer0calls = 0;
38 uint32_t numSWIcalls = 0;
39 extern uint32_t numRXA;
40 uint16_t UARTPrint = 0;
41 uint16_t LEDdisplaynum = 0;
42
43
44 void main(void)
45 {
46     // PLL, WatchDog, enable Peripheral Clocks
47     // This example function is found in the F2837xD_SysCtrl.c file.
48     InitSysCtrl();
49
50     InitGpio();
51
52     // Blue LED on LaunchPad
53     GPIO_SetupPinMux(31, GPIO_MUX_CPU1, 0);
54     GPIO_SetupPinOptions(31, GPIO_OUTPUT, GPIO_PUSHPULL);
55     GpioDataRegs.GPASET.bit.GPIO31 = 1;
56
57     // Red LED on LaunchPad
58     GPIO_SetupPinMux(34, GPIO_MUX_CPU1, 0);
59     GPIO_SetupPinOptions(34, GPIO_OUTPUT, GPIO_PUSHPULL);
60     GpioDataRegs.GPBSET.bit.GPIO34 = 1;
61
62     // LED1 and PWM Pin
63     GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 5);
64     GPIO_SetupPinOptions(22, GPIO_OUTPUT, GPIO_PUSHPULL);
65
66     // LED2
67     GPIO_SetupPinMux(94, GPIO_MUX_CPU1, 0);
68     GPIO_SetupPinOptions(94, GPIO_OUTPUT, GPIO_PUSHPULL);
69     GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
70
71     // LED3
72     GPIO_SetupPinMux(95, GPIO_MUX_CPU1, 0);
73     GPIO_SetupPinOptions(95, GPIO_OUTPUT, GPIO_PUSHPULL);
74     GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
75
76     // LED4
77     GPIO_SetupPinMux(97, GPIO_MUX_CPU1, 0);
78     GPIO_SetupPinOptions(97, GPIO_OUTPUT, GPIO_PUSHPULL);
79     GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
80
81     // LED5
82     GPIO_SetupPinMux(111, GPIO_MUX_CPU1, 0);
83     GPIO_SetupPinOptions(111, GPIO_OUTPUT, GPIO_PUSHPULL);
84     GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
85
86     // LED6

```

```

87  GPIO_SetupPinMux(130, GPIO_MUX_CPU1, 0);
88  GPIO_SetupPinOptions(130, GPIO_OUTPUT, GPIO_PUSHPULL);
89  GpioDataRegs.GPECLEAR.bit.GPIO130 = 1;
90
91  // LED7
92  GPIO_SetupPinMux(131, GPIO_MUX_CPU1, 0);
93  GPIO_SetupPinOptions(131, GPIO_OUTPUT, GPIO_PUSHPULL);
94  GpioDataRegs.GPECLEAR.bit.GPIO131 = 1;
95
96  // LED8
97  GPIO_SetupPinMux(25, GPIO_MUX_CPU1, 0);
98  GPIO_SetupPinOptions(25, GPIO_OUTPUT, GPIO_PUSHPULL);
99  GpioDataRegs.GPACLEAR.bit.GPIO25 = 1;
100
101  // LED9
102  GPIO_SetupPinMux(26, GPIO_MUX_CPU1, 0);
103  GPIO_SetupPinOptions(26, GPIO_OUTPUT, GPIO_PUSHPULL);
104  GpioDataRegs.GPACLEAR.bit.GPIO26 = 1;
105
106  // LED10
107  GPIO_SetupPinMux(27, GPIO_MUX_CPU1, 0);
108  GPIO_SetupPinOptions(27, GPIO_OUTPUT, GPIO_PUSHPULL);
109  GpioDataRegs.GPACLEAR.bit.GPIO27 = 1;
110
111  // LED11
112  GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
113  GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
114  GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
115
116  // LED12
117  GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
118  GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
119  GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
120
121  // LED13
122  GPIO_SetupPinMux(157, GPIO_MUX_CPU1, 0);
123  GPIO_SetupPinOptions(157, GPIO_OUTPUT, GPIO_PUSHPULL);
124  GpioDataRegs.GPECLEAR.bit.GPIO157 = 1;
125
126  // LED14
127  GPIO_SetupPinMux(158, GPIO_MUX_CPU1, 0);
128  GPIO_SetupPinOptions(158, GPIO_OUTPUT, GPIO_PUSHPULL);
129  GpioDataRegs.GPECLEAR.bit.GPIO158 = 1;
130
131  // LED15
132  GPIO_SetupPinMux(159, GPIO_MUX_CPU1, 0);
133  GPIO_SetupPinOptions(159, GPIO_OUTPUT, GPIO_PUSHPULL);
134  GpioDataRegs.GPECLEAR.bit.GPIO159 = 1;
135
136  // LED16
137  GPIO_SetupPinMux(160, GPIO_MUX_CPU1, 0);
138  GPIO_SetupPinOptions(160, GPIO_OUTPUT, GPIO_PUSHPULL);
139  GpioDataRegs.GPFCLEAR.bit.GPIO160 = 1;
140
141  //WIZNET Reset
142  GPIO_SetupPinMux(0, GPIO_MUX_CPU1, 0);
143  GPIO_SetupPinOptions(0, GPIO_OUTPUT, GPIO_PUSHPULL);
144  GpioDataRegs.GPASET.bit.GPIO0 = 1;
145
146  //ESP8266 Reset
147  GPIO_SetupPinMux(1, GPIO_MUX_CPU1, 0);
148  GPIO_SetupPinOptions(1, GPIO_OUTPUT, GPIO_PUSHPULL);
149  GpioDataRegs.GPASET.bit.GPIO1 = 1;
150
151  //SPIRAM CS Chip Select
152  GPIO_SetupPinMux(19, GPIO_MUX_CPU1, 0);
153  GPIO_SetupPinOptions(19, GPIO_OUTPUT, GPIO_PUSHPULL);
154  GpioDataRegs.GPASET.bit.GPIO19 = 1;
155
156  //DRV8874 #1 DIR Direction
157  GPIO_SetupPinMux(29, GPIO_MUX_CPU1, 0);
158  GPIO_SetupPinOptions(29, GPIO_OUTPUT, GPIO_PUSHPULL);
159  GpioDataRegs.GPASET.bit.GPIO29 = 1;
160
161  //DRV8874 #2 DIR Direction
162  GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
163  GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
164  GpioDataRegs.GPBSET.bit.GPIO32 = 1;
165
166  //DAN28027 CS Chip Select
167  GPIO_SetupPinMux(9, GPIO_MUX_CPU1, 0);
168  GPIO_SetupPinOptions(9, GPIO_OUTPUT, GPIO_PUSHPULL);
169  GpioDataRegs.GPASET.bit.GPIO9 = 1;
170
171  //MPU9250 CS Chip Select
172  GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0);
173  GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSHPULL);
174  GpioDataRegs.GPCSET.bit.GPIO66 = 1;
175

```

```

176 //WIZNET CS Chip Select
177 GPIO_SetupPinMux(125, GPIO_MUX_CPU1, 0);
178 GPIO_SetupPinOptions(125, GPIO_OUTPUT, GPIO_PUSH_PULL);
179 GpioDataRegs.GPDSSET.bit.GPIO125 = 1;
180
181 //PushButton 1
182 GPIO_SetupPinMux(4, GPIO_MUX_CPU1, 0);
183 GPIO_SetupPinOptions(4, GPIO_INPUT, GPIO_PULLUP);
184
185 //PushButton 2
186 GPIO_SetupPinMux(5, GPIO_MUX_CPU1, 0);
187 GPIO_SetupPinOptions(5, GPIO_INPUT, GPIO_PULLUP);
188
189 //PushButton 3
190 GPIO_SetupPinMux(6, GPIO_MUX_CPU1, 0);
191 GPIO_SetupPinOptions(6, GPIO_INPUT, GPIO_PULLUP);
192
193 //PushButton 4
194 GPIO_SetupPinMux(7, GPIO_MUX_CPU1, 0);
195 GPIO_SetupPinOptions(7, GPIO_INPUT, GPIO_PULLUP);
196
197 //Joy Stick Pushbutton
198 GPIO_SetupPinMux(8, GPIO_MUX_CPU1, 0);
199 GPIO_SetupPinOptions(8, GPIO_INPUT, GPIO_PULLUP);
200
201
202 // Clear all interrupts and initialize PIE vector table:
203 // Disable CPU interrupts
204 DINT;
205
206 // Initialize the PIE control registers to their default state.
207 // The default state is all PIE interrupts disabled and flags
208 // are cleared.
209 // This function is found in the F2837xD_PieCtrl.c file.
210 InitPieCtrl();
211
212 // Disable CPU interrupts and clear all CPU interrupt flags:
213 IER = 0x0000;
214 IFR = 0x0000;
215
216 // Initialize the PIE vector table with pointers to the shell Interrupt
217 // Service Routines (ISR).
218 // This will populate the entire table, even if the interrupt
219 // is not used in this example. This is useful for debug purposes.
220 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
221 // This function is found in F2837xD_PieVect.c.
222 InitPieVectTable();
223
224 // Interrupts that are used in this example are re-mapped to
225 // ISR functions found within this project
226 EALLOW; // This is needed to write to EALLOW protected registers
227 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
228 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
229 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
230 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
231 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
232 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
233 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
234 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
235 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
236 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
237 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
238
239 PieVectTable.EMIF_ERROR_INT = &SWI_isr;
240 EDIS; // This is needed to disable write to EALLOW protected registers
241
242
243 // Initialize the CpuTimers Device Peripheral. This function can be
244 // found in F2837xD_CpuTimers.c
245 InitCpuTimers();
246
247 // Configure CPU-Timer 0, 1, and 2 to interrupt every given period:
248 // 200MHz CPU Freq, Period (in uSeconds)
249 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);
250 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 20000);
251 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000);
252
253 // Enable CpuTimer Interrupt bit TIE
254 CpuTimer0Regs.TCR.all = 0x4000;
255 CpuTimer1Regs.TCR.all = 0x4000;
256 CpuTimer2Regs.TCR.all = 0x4000;
257
258 init_serialSCIA(&SerialA, 115200);
259 // init_serialSCIC(&SerialC, 115200);
260 // init_serialSCID(&SerialD, 115200);
261
262 // To know which Registry needs EALLOW, this is in the condensed registries and then
263 // "11.16.2 ADC_REGS Registers" or "15.15.2 EPWM_REGS Registers"
264 EALLOW;

```

```

265 // TBCTL
266 // Count up Mode
267 // Free Soft emulation = Free Run
268 // Do not load time time-base counter from the time-base phase registry
269 // Clock divide = 1
270 EPwm12Regs.TBCTL.bit.FREE_SOFT = 0b10;
271 EPwm12Regs.TBCTL.bit.CLKDIV = 0;
272 EPwm12Regs.TBCTL.bit.CTRMODE = 0;
273 EPwm12Regs.TBCTL.bit.PHSEN = 0;
274
275
276 // TBCTR
277 // Start Timer at zero
278 EPwm12Regs.TBCTR = 0;
279
280
281 // TBPRD
282 // Set period of the PWM to 5kHz (200 us)
283 // Counter frequency is 50MHz
284 EPwm12Regs.TBPRD = 10000;
285
286 // CMPA
287 // Duty cycle at 50%, TBPRD / 2
288 EPwm12Regs.CMPA.bit.CMPA = 5000;
289
290 EPwm12Regs.AQCTLA.bit.CAU = 0b01; // PWM12A output pin is set low when CMPA is reached
291 EPwm12Regs.AQCTLA.bit.ZRO = 0b10; // the pin be set high when the TBCTR register is zero
292
293 EPwm12Regs.TBPHS.bit.TBPHS = 0;
294
295 GpioCtrlRegs.GPAPUD.bit.GPIO22 = 1; // For EPW12A
296 EDIS;
297
298
299 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
300 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
301 // to CPU-Timer 2: int 12 is for the SWI.
302 IER |= M_INT1;
303 IER |= M_INT8; // SCIC SCID
304 IER |= M_INT9; // SCIA
305 IER |= M_INT12;
306 IER |= M_INT13;
307 IER |= M_INT14;
308
309 // Enable TINT0 in the PIE: Group 1 interrupt 7
310 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
311 // Enable SWI in the PIE: Group 12 interrupt 9
312 PieCtrlRegs.PIEIER12.bit.INTx9 = 1;
313
314 // Enable global Interrupts and higher priority real-time debug events
315 EINT; // Enable Global interrupt INTM
316 ERTM; // Enable Global realtime interrupt DBGM
317
318
319 // IDLE loop. Just sit and loop forever (optional):
320 while(1)
321 {
322     if (UARTPrint == 1) {
323         serial_printf(&SerialA, "Num Timer2:%ld Num SerialRX: %ld\r\n", CpuTimer2.InterruptCount, numRXA);
324         UARTPrint = 0;
325     }
326 }
327 }
328
329
330 // SWI_isr, Using this interrupt as a Software started interrupt
331 __interrupt void SWI_isr(void) {
332
333     // These three lines of code allow SWI_isr, to be interrupted by other interrupt functions
334     // making it lower priority than all other Hardware interrupts.
335     PieCtrlRegs.PIEACK.all = PIEACK_GROUP12;
336     asm("NOP"); // Wait one cycle
337     EINT; // Clear INTM to enable interrupts
338
339
340
341 // Insert SWI ISR Code here.....
342
343 numSWIcalls++;
344
345 DINT;
346 }
347
348
349 // cpu_timer0_isr - CPU Timer0 ISR
350 __interrupt void cpu_timer0_isr(void)
351 {
352     CpuTimer0.InterruptCount++;
353 }

```

```

354
355     numTimer0calls++;
356
357     //     if ((numTimer0calls%50) == 0) {
358     //         PieCtrlRegs.PIEIFR12.bit.INTx9 = 1; // Manually cause the interrupt for the SWI
359     //     }
360
361     if ((numTimer0calls%250) == 0) {
362 //         displayLEDletter(LEDdisplaynum);
363         LEDdisplaynum++;
364         if (LEDdisplaynum == 0xFFFF) { // prevent roll over exception
365             LEDdisplaynum = 0;
366         }
367     }
368
369     // Blink LaunchPad Red LED
370     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
371
372     // Acknowledge this interrupt to receive more interrupts from group 1
373     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
374 }
375
376 // cpu_timer1_isr - CPU Timer1 ISR
377 __interrupt void cpu_timer1_isr(void)
378 {
379
380
381     CpuTimer1.InterruptCount++;
382 }
383
384 // cpu_timer2_isr CPU Timer2 ISR
385 __interrupt void cpu_timer2_isr(void)
386 {
387
388
389     // Blink LaunchPad Blue LED
390     GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
391
392     CpuTimer2.InterruptCount++;
393
394     if ((CpuTimer2.InterruptCount % 50) == 0) {
395         UARTPrint = 1;
396     }
397 }

```